МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

(Университет ИТМО)

ОТЧЁТ

по лабораторной работе №3

«Матрицы в 3D-графике»

по дисциплине «Практическая линейная алгебра»

Выполнили:

студент группы R3243 Осипов А. П.

студент группы R3281 Глазков В. Н.

студент группы R3235 Прохоревич И. Б.

Промежуточная версия: задания 1–5

Проверил:

Догадин Е. В.

Санкт-Петербург

2026

Содержание

Введение **3**

Исходные данные **3**

1 Задание 1. Отрисовка куба и другой фигуры **4**

2 Задание 2. Масштабирование куба **6**

3 Задание 3. Перенос и порядок преобразований **8**

4 Задание 4. Вращение кубика **13**

5 Задание 5. Вращение кубика вокруг вершины **26**

Заключение по заданиям 1–5 **32**

Приложение А. Код блока А **33**

Приложение Б. Код заданий 4–5 **37**

Введение

В отчёте представлены задания 1–5 лабораторной работы №3: отрисовка фигур в однородных координатах, масштабирование, перенос и вращение куба, сравнение порядка применения преобразований, поворот вокруг оси через вершину. Расчёты выполнены на Python с использованием NumPy, Matplotlib и SciPy. Все преобразования координат реализованы умножением на матрицы. Задания 6–7 о камере и перспективе в этой версии ещё не представлены.

Исходные данные

- Исходный куб из методички: координаты вершин по каждой оси равны -1 или 1, длина ребра — 2.
- Вершины тетраэдра, выбранные Альбертом: $A = (-2, -2, 0)$, $B = (4, -4, 0)$, $C = (0, 8, 0)$, $D = (1, 1, 10)$.
- Коэффициенты масштабирования, выбранные Альбертом для задания 2: $(s_{1x}, s_{1y}, s_{1z}) = (2, 4, 10)$ и $(s_{2x}, s_{2y}, s_{2z}) = (0.5, 1, 0.1)$.
- Векторы переноса, выбранные Альбертом для задания 3: $t_1 = (5, 5, 2)$ и $t_2 = (0, 3, 10)$.
- Для сравнения порядка преобразований используются $T = T_1$ и $S = S_1$.

1 Задание 1. Отрисовка куба и другой фигуры

Однородные координаты и устройство кода

Каждая вершина хранится в столбце матрицы `vertices` как $\widetilde{p} = (x, y, z, w)^T$. Для выбранных исходных фигур $w = 1$. Переход к декартовым координатам при $w \neq 0$ имеет вид

$$p = \begin{pmatrix} x/w \\ y/w \\ z/w \end{pmatrix}.$$

Например, $(4, 6, 2, 2)^T$ задаёт точку $(2, 3, 1)^T$. Строка программы

```
points = (vertices[:3, :] / vertices[3, :]).T
```

выполняет это деление для всех вершин. Транспонирование `.T` переводит матрицу размера $3 \times N$ в $N \times 3$: отрисовщику нужны вершины по строкам. Каждая строка `faces` задаёт индексы вершин одной грани в порядке обхода. Индексация начинается с нуля. `points[faces]` собирает координаты граней, а `Poly3DCollection` отображает их в пространстве. Использована ортографическая проекция `proj_type="ortho"`.

Исходный куб

Матрицы из Python-листинга методички:

$$V = \begin{pmatrix} -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix},$$

$$F = \begin{pmatrix} 0 & 1 & 5 & 4 \\ 1 & 2 & 6 & 5 \\ 2 & 3 & 7 & 6 \\ 3 & 0 & 4 & 7 \\ 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{pmatrix}.$$

У грани $[0, 1, 2, 3]$ все вершины имеют $z = -1$: это нижняя грань куба. Центр куба находится в начале координат, длина ребра равна 2.

Тетраэдр

Вместо куба заданы вершины $A = (-2, -2, 0)$, $B = (4, -4, 0)$, $C = (0, 8, 0)$, $D = (1, 1, 10)$:

$$V_{\text{tetra}} = \begin{pmatrix} -2 & 4 & 0 & 1 \\ -2 & -4 & 8 & 1 \\ 0 & 0 & 0 & 10 \\ 1 & 1 & 1 & 1 \end{pmatrix}, \qquad F_{\text{tetra}} = \begin{pmatrix} 0 & 2 & 1 \\ 0 & 1 & 3 \\ 1 & 2 & 3 \\ 2 & 0 & 3 \end{pmatrix}.$$

Первые три вершины лежат в плоскости $z = 0$, четвёртая — вне этой плоскости. Вместо шести четырёхугольных граней получены четыре треугольные. Пределы осей расширены под выбранные координаты; пропорции области построения согласованы с диапазонами осей, чтобы единичный масштаб по всем осям совпадал. Куб и тетраэдр показаны на рисунке 1.

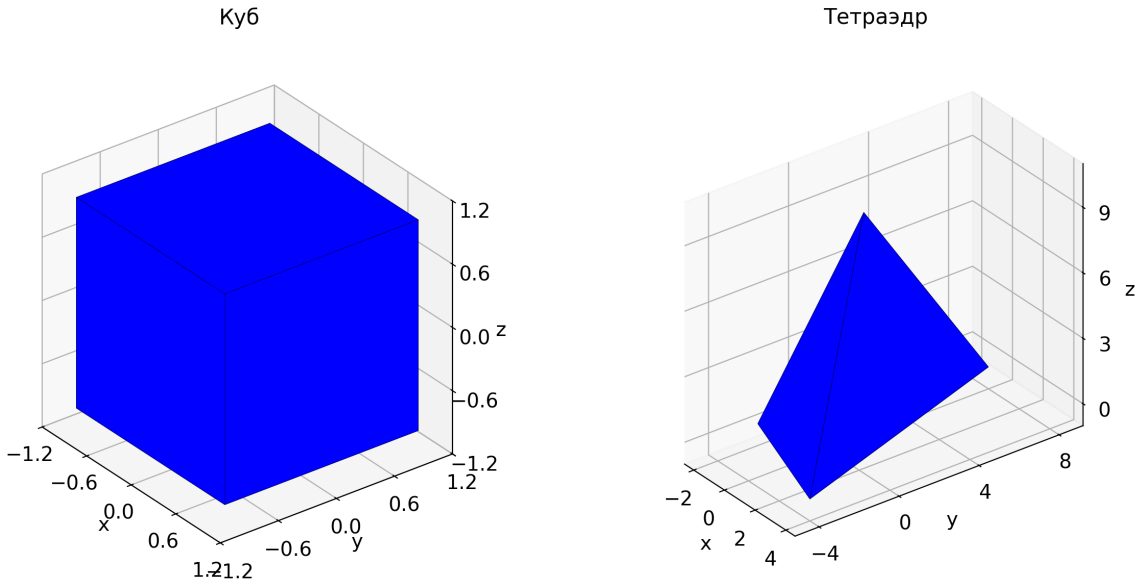


Рисунок 1 – Куб из методички и тетраэдр с выбранными вершинами

Вывод по заданию 1

Матрица 3×3 не может перенести начало координат в другую точку, поскольку произведение любой матрицы на нулевой вектор равно нулевому вектору. Четвёртая координата позволяет описать перенос матричным умножением; перед отрисовкой однородные координаты переводятся в декартовы делением на w.

2 Задание 2. Масштабирование куба

Общая структура матрицы

Масштабирование по координатным осям задаётся матрицей

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} s_x x \\ s_y y \\ s_z z \\ 1 \end{pmatrix}.$$

Матрицы масштаба отдельно по x, y, z получаются подстановкой $(s_x, 1, 1)$, $(1, s_y, 1)$, $(1, 1, s_z)$ соответственно. Все вершины преобразуются одновременно: $V' = SV$, в Python — `S @ vertices_cube`.

Выбранные матрицы и их композиция

$$S_1 = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S_2 = \begin{pmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Сначала применяется S_1, затем S_2. Для векторов-столбцов последняя применяемая матрица стоит слева:

$$S_2(S_1 V) = (S_2 S_1)V, \qquad S_2 S_1 = \text{diag}(0.5 \cdot 2,\ 1 \cdot 4,\ 0.1 \cdot 10,\ 1) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Отдельные матрицы S_1 и S_2 применяются к исходному кубу, а третий результат соответствует их последовательному применению.

Таблица 1 – Размеры куба после масштабирования

Преобразование	Размер по x	Размер по y	Размер по z
Исходный куб	2	2	2
S_1	4	8	20
S_2	1	2	0.2
$S_2 S_1$	2	8	2

Например, для вершины $p = (-1, -1, -1, 1)^T$:

$$S_1 p = (-2, -4, -10, 1)^T, \quad S_2 p = (-0.5, -1, -0.1, 1)^T, \quad S_2 S_1 p = (-1, -4, -1, 1)^T.$$

Результаты приведены на рисунке 2. У панелей разные пределы осей, поэтому абсолютные размеры нужно сравнивать по численным отметкам и таблице. Внутри каждой панели единичный масштаб одинаков по трём осям.

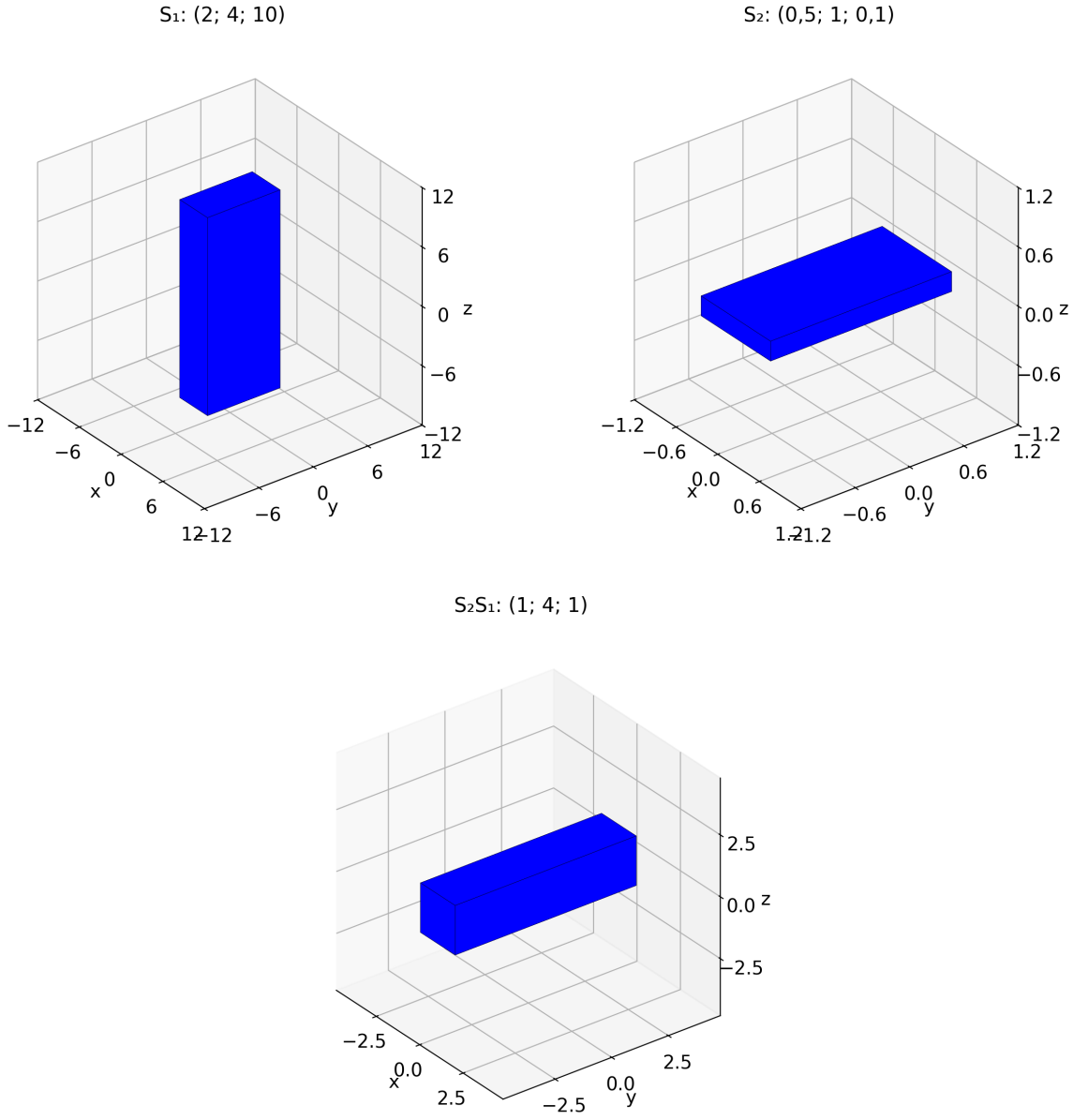


Рисунок 2 – Два масштаба и их композиция

Вывод по заданию 2

При последовательном масштабировании коэффициенты по каждой оси перемножаются. По оси x получаем $2 \cdot 0.5 = 1$, а по оси z — $10 \cdot 0.1 = 1$: второе преобразование компенсирует первое, поэтому размеры по этим осям возвращаются к исходным. По оси y произведение равно $4 \cdot 1 = 4$, поэтому растяжение в четыре раза сохраняется. Результат композиции — прямоугольный параллелепипед с размерами $(2, 8, 2)$.

3 Задание 3. Перенос и порядок преобразований

Общая структура матрицы переноса

$$T(a,b,c) = \begin{pmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad T \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x+a \\ y+b \\ z+c \\ 1 \end{pmatrix}.$$

Переносы отдельно по x, y, z задаются $T(a,0,0)$, $T(0,b,0)$, $T(0,0,c)$ соответственно. Последний столбец добавляет смещение к координатам.

Выбранные переносы и композиция

$$T_1 = \begin{pmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 5 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad T_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 10 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

При последовательном применении векторы переноса складываются:

$$T_2 T_1 = T(5+0,\ 5+3,\ 2+10) = \begin{pmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 8 \\ 0 & 0 & 1 & 12 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Центры образов куба равны $(5, 5, 2)$, $(0, 3, 10)$ и $(5, 8, 12)$ соответственно. Для исходной вершины $p = (-1, -1, -1, 1)^T$:

$$T_1 p = (4, 4, 1, 1)^T, \quad T_2 p = (-1, 2, 9, 1)^T, \quad T_2 T_1 p = (4, 7, 11, 1)^T.$$

Во всех трёх случаях размеры равны $(2, 2, 2)$. Результаты показаны на рисунке 3. Серым пунктиром обозначен исходный куб; у трёх панелей одинаковые пределы осей.

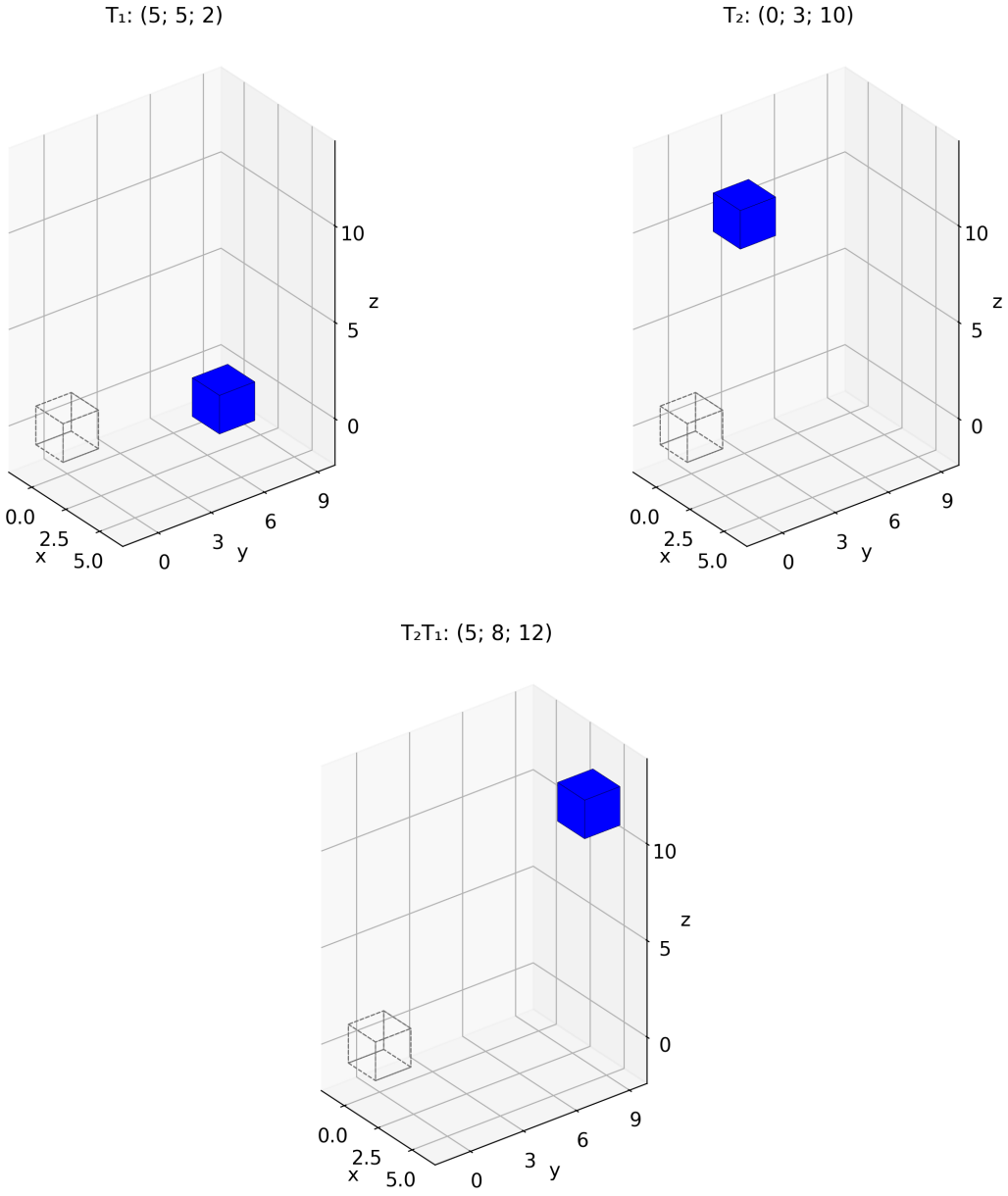


Рисунок 3 – Два переноса и их композиция; исходный куб показан контуром

Сравнение TS и ST

Для сравнения выбраны $T = T_1$ и $S = S_1$:

$$TS = \begin{pmatrix} 2 & 0 & 0 & 5 \\ 0 & 4 & 0 & 5 \\ 0 & 0 & 10 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad ST = \begin{pmatrix} 2 & 0 & 0 & 10 \\ 0 & 4 & 0 & 20 \\ 0 & 0 & 10 & 20 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

При TS точка сначала масштабируется, затем переносится:

$$(x, y, z) \longmapsto (2x + 5,\ 4y + 5,\ 10z + 2).$$

При ST точка сначала переносится, затем масштабируется:

$$(x, y, z) \longmapsto (2(x + 5),\ 4(y + 5),\ 10(z + 2)) = (2x + 10,\ 4y + 20,\ 10z + 20).$$

Например, $TSp = (3, 1, -8, 1)^T$, а $STp = (8, 16, 10, 1)^T$. Разность декартовых координат равна $(5, 15, 18)$ для каждой вершины.

Таблица 2 – Центры и размеры образов TS и ST

Преобразование	Центр образа	Размеры по x, y, z
TS	$(5, 5, 2)$	$(4, 8, 20)$
ST	$(10, 20, 20)$	$(4, 8, 20)$

Результаты неэквивалентны: матрицы отличаются последним столбцом и дают разные координаты образов. Сравнение приведено на рисунке 4.

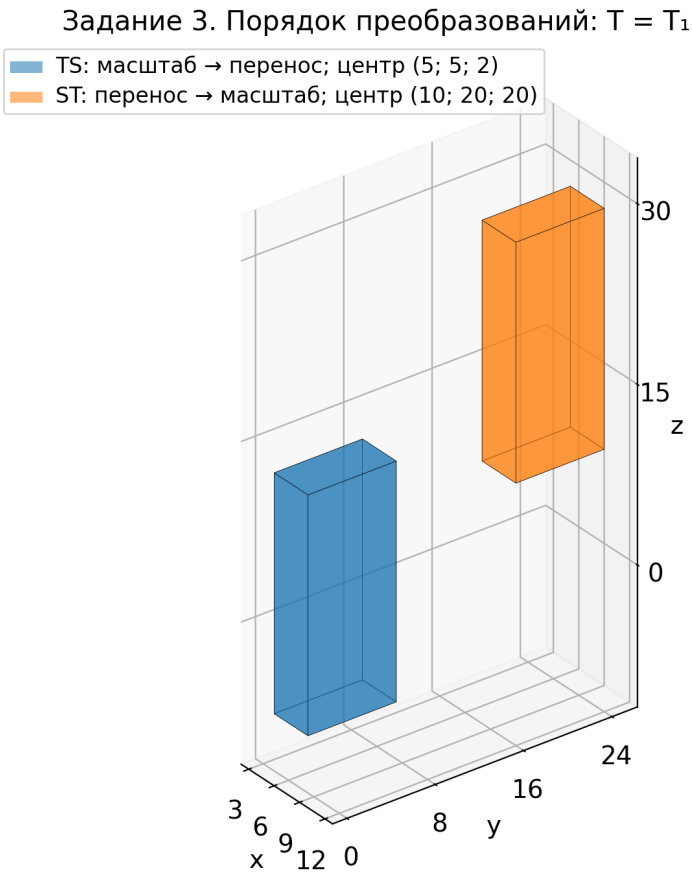


Рисунок 4 – Сравнение порядка применения масштабирования и переноса

Вывод по заданию 3

Перенос добавляет один и тот же вектор ко всем вершинам, поэтому разности их координат и размеры куба не меняются. Два последовательных переноса равносильны одному переносу на сумму векторов: $(5, 8, 12)$.

В общем случае умножение матриц некоммутативно: порядок преобразований может менять результат. Для выбранных матриц $TS \neq ST$. В TS масштаб применяется до переноса, а в ST масштабируется и добавленное смещение: $(5, 5, 2)$ превращается в $(2 \cdot 5, 4 \cdot 5, 10 \cdot 2) = (10, 20, 20)$. Поэтому положения образов различаются. Размеры в обоих случаях одинаковы, $(4, 8, 20)$, поскольку применяется один и тот же масштаб, а перенос размеры не меняет.

Определители произведений совпадают:

$$\det(TS) = \det(T)\det(S) = \det(S)\det(T) = \det(ST) = 1 \cdot 80 = 80.$$

Это следует из коммутативности умножения чисел и означает одинаковое изменение объёма — в 80 раз. Само по себе равенство определителей не гарантирует одинаковых размеров или положения фигур.

4 Задание 4. Вращение кубика

Постановка задачи

Требуется повернуть кубик вокруг произвольной оси, проходящей через его центр. Ось вращения задается вектором

$$v = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix},$$

а угол поворота — величиной θ. Матрица поворота определяется через матричную экспоненту:

$$R_v(\theta) = e^{J\theta}, \qquad J = \frac{1}{\|v\|} \begin{bmatrix} 0 & -v_z & v_y & 0 \\ v_z & 0 & -v_x & 0 \\ -v_y & v_x & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Матрица J имеет размер 4×4, поскольку вершины кубика заданы в однородных координатах. Последняя строка и последний столбец матрицы J нулевые, поэтому в матричной экспоненте в правом нижнем углу стоит единица, а четвертая координата остается равной единице. Следовательно, преобразование является аффинным.

Исходные данные

Кубик задан в однородных координатах. Его вершины являются столбцами матрицы

$$V = \begin{bmatrix} -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

Центр кубика совпадает с началом координат, поэтому ось вращения, проходящая через центр кубика, проходит через точку $(0, 0, 0)$, и отдельная матрица переноса не требуется.

Общая структура матрицы поворота

Пусть

$$u = \frac{v}{\|v\|} = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}, \qquad \|u\| = 1$$

— нормированный вектор оси вращения. Кососимметрическая матрица, соответствующая этому вектору, имеет вид

$$K = \begin{bmatrix} 0 & -u_z & u_y & 0 \\ u_z & 0 & -u_x & 0 \\ -u_y & u_x & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Для нормированного вектора u выполняется свойство

$$K^3 = -K.$$

Поэтому матричная экспонента приводится к формуле Родригеса:

$$R_u(\theta) = e^{K\theta} = I + \sin\theta\, K + (1 - \cos\theta) K^2.$$

Для компактной записи положим $c = \cos\theta$, $s = \sin\theta$, $d = 1 - c$. Матрица поворота вокруг произвольной оси, проходящей через начало координат, имеет вид

$$R_u(\theta) = \begin{bmatrix} c + u_x^2 d & u_x u_y d - u_z s & u_x u_z d + u_y s & 0 \\ u_y u_x d + u_z s & c + u_y^2 d & u_y u_z d - u_x s & 0 \\ u_z u_x d - u_y s & u_z u_y d + u_x s & c + u_z^2 d & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

В программе матрица вычисляется напрямую по формуле $R_v(\theta) = e^{J\theta}$ с помощью `scipy.linalg.expm`; формула Родригеса приведена выше для аналитической записи этой же матрицы.

Матрицы поворота вокруг координатных осей

Частными случаями общей формулы являются повороты вокруг осей x, y и z.

Поворот вокруг оси x:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Поворот вокруг оси y:

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Поворот вокруг оси z:

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Эти матрицы получаются из общей формулы при выборе единичных векторов

$$e_x = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad e_y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad e_z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Выбранные оси и углы поворота

Для выполнения задания выбраны две произвольные оси вращения и два угла поворота.

Первая ось:

$$v_1 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad \theta_1 = \frac{\pi}{2}, \qquad u_1 = \frac{v_1}{\|v_1\|} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}.$$

Ось v_1 проходит через центр кубика и направлена вдоль диагонали плоскости xy.

Вторая ось:

$$v_2 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \qquad \theta_2 = \frac{\pi}{3}, \qquad u_2 = \frac{v_2}{\|v_2\|} = \begin{bmatrix} 0 \\ \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}.$$

Ось v_2 проходит через центр кубика и направлена вдоль диагонали плоскости yz.

Матрица первого поворота

Для оси v_1 и угла $\theta_1 = \frac{\pi}{2}$ имеем

$$\sin\frac{\pi}{2} = 1, \qquad \cos\frac{\pi}{2} = 0.$$

Подстановка в общую формулу дает

$$R_1 = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\approx \begin{bmatrix} 0.500000 & 0.500000 & 0.707107 & 0 \\ 0.500000 & 0.500000 & -0.707107 & 0 \\ -0.707107 & 0.707107 & 0.000000 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Эта матрица выполняет поворот кубика на 90° вокруг оси, направленной вдоль вектора v_1.

Проверим, что точка на оси вращения остается неподвижной. Возьмем точку

$$p = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix}.$$

Тогда

$$R_1 p = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = p.$$

Следовательно, прямая, проходящая через начало координат вдоль вектора v_1, действительно является осью вращения.

Матрица второго поворота

Для оси v_2 и угла $\theta_2 = \frac{\pi}{3}$ имеем

$$\sin\frac{\pi}{3} = \frac{\sqrt{3}}{2}, \qquad \cos\frac{\pi}{3} = \frac{1}{2}.$$

После подстановки в общую формулу получаем

$$R_2 = \begin{bmatrix} \frac{1}{2} & -\frac{\sqrt{6}}{4} & \frac{\sqrt{6}}{4} & 0 \\ \frac{\sqrt{6}}{4} & \frac{3}{4} & \frac{1}{4} & 0 \\ -\frac{\sqrt{6}}{4} & \frac{1}{4} & \frac{3}{4} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\approx \begin{bmatrix} 0.500000 & -0.612372 & 0.612372 & 0 \\ 0.612372 & 0.750000 & 0.250000 & 0 \\ -0.612372 & 0.250000 & 0.750000 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Эта матрица выполняет поворот кубика на $60°$ вокруг оси, направленной вдоль вектора v_2.

Проверка неподвижности оси:

$$p = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \qquad R_2 p = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} = p.$$

Следовательно, ось вращения задана корректно.

Композиция двух поворотов

Поскольку вершины кубика рассматриваются как столбцы матрицы V, преобразование применяется умножением матрицы слева:

$$V' = RV.$$

Если сначала выполняется поворот R_1, а затем R_2, итоговая матрица имеет вид

$$M_{1\to 2} = R_2 R_1.$$

Если сначала выполняется поворот R_2, а затем R_1, итоговая матрица имеет вид

$$M_{2\to 1} = R_1 R_2.$$

В общем случае повороты в трехмерном пространстве не коммутируют, поэтому $R_2 R_1 \neq R_1 R_2$.

Композиция $M_{1\to 2} = R_2 R_1$

Сначала выполняется поворот вокруг оси v_1 на угол $\theta_1 = \frac{\pi}{2}$, затем — поворот вокруг оси v_2 на угол $\theta_2 = \frac{\pi}{3}$. Численно:

$$M_{1\to 2} \approx \begin{bmatrix} -0.489199 & 0.376826 & 0.786566 & 0 \\ 0.504410 & 0.857963 & -0.097317 & 0 \\ -0.711516 & 0.349144 & -0.609789 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Результирующее преобразование также является поворотом вокруг некоторой оси. Нормированный вектор эквивалентной оси вращения приближенно равен

$$u_{1\to 2} \approx \begin{bmatrix} 0.2847 \\ 0.9552 \\ 0.0813 \end{bmatrix}.$$

Композиция $M_{2\to1} = R_1 R_2$

Сначала выполняется поворот вокруг оси v_2 на угол $\theta_2 = \frac{\pi}{3}$, затем — поворот вокруг оси v_1 на угол $\theta_1 = \frac{\pi}{2}$. Численно:

$$M_{2\to1} \approx \begin{bmatrix} 0.123174 & 0.245590 & 0.961516 & 0 \\ 0.989199 & -0.107963 & -0.099144 & 0 \\ 0.079459 & 0.963343 & -0.256236 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Нормированный вектор эквивалентной оси вращения приближенно равен

$$u_{2\to1} \approx \begin{bmatrix} 0.6774 \\ 0.5624 \\ 0.4741 \end{bmatrix}.$$

Демонстрация некоммутативности поворотов

Сравним полученные матрицы:

$$M_{1\to2} = R_2 R_1, \qquad M_{2\to1} = R_1 R_2.$$

Видно, что $M_{1\to2} \neq M_{2\to1}$. Например, элемент в первой строке и первом столбце:

$$(M_{1\to2})_{11} \approx -0.489199, \qquad (M_{2\to1})_{11} \approx 0.123174.$$

Следовательно, порядок применения поворотов существенно влияет на итоговое положение кубика: последовательные повороты вокруг различных осей в трехмерном пространстве не коммутируют.

Результаты выполнения программы

На каждом графике показаны исходный кубик (синий, полупрозрачный), кубик после применения соответствующей матрицы (красный), пунктирная ось вращения и вектор, коллинеарный этой оси. Для композиций на графиках показана эквивалентная ось результирующего поворота.

После применения любой из матриц четвертая однородная координата остается равной единице, поэтому в таблицах приведены только декартовы координаты x, y и z.

Поворот вокруг оси v_1

Применяется матрица R_1. Ось вращения — $v_1 = (1, 1, 0)^T$, угол поворота — $\theta_1 = \frac{\pi}{2}$. Ожидаемый результат: кубик повернут на $90°$ вокруг диагонали плоскости xy.

Результат показан на рисунке 5, координаты вершин приведены в таблице 3.

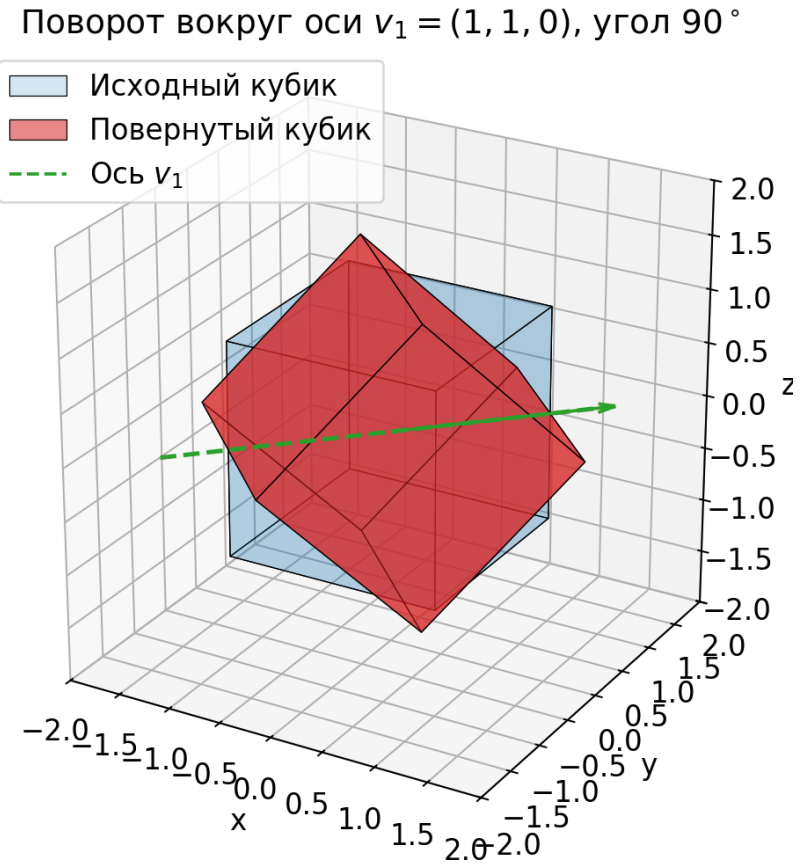


Рисунок 5 – Поворот на $90°$ вокруг оси $v_1 = (1, 1, 0)$

Таблица 3 – Координаты вершин после применения R_1

№ вершины	x	y	z
1	-1.707	-0.293	0.000
2	-0.707	0.707	-1.414
3	0.293	1.707	0.000
4	-0.707	0.707	1.414
5	-0.293	-1.707	0.000
6	0.707	-0.707	-1.414
7	1.707	0.293	0.000
8	0.707	-0.707	1.414

Поворот вокруг оси v_2

Применяется матрица R_2. Ось вращения — $v_2 = (0, 1, 1)^T$, угол поворота — $\theta_2 = \frac{\pi}{3}$. Ожидаемый результат: кубик повернут на $60°$ вокруг диагонали плоскости yz.

Результат показан на рисунке 6, координаты вершин приведены в таблице 4.

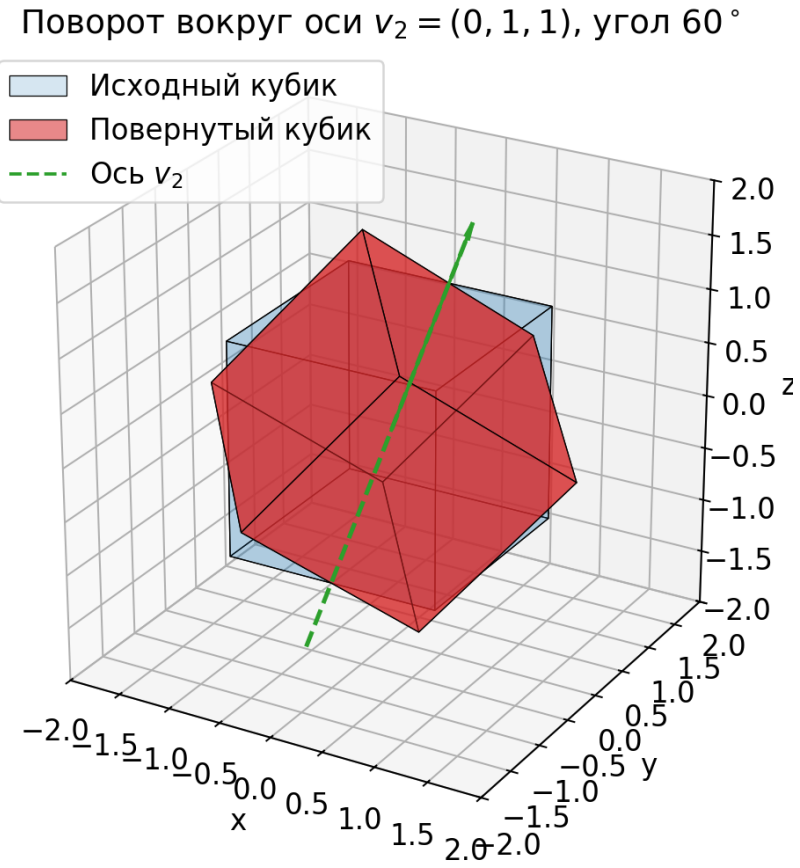


Рисунок 6 – Поворот на $60°$ вокруг оси $v_2 = (0, 1, 1)$

Таблица 4 – Координаты вершин после применения R_2

№ вершины	x	y	z
1	-0.500	-1.612	-0.388
2	0.500	-0.388	-1.612
3	-0.725	1.112	-1.112
4	-1.725	-0.112	0.112
5	0.725	-1.112	1.112
6	1.725	0.112	-0.112
7	0.500	1.612	0.388
8	-0.500	0.388	1.612

Композиция $M_{1\to2} = R_2 R_1$

Порядок преобразований: сначала поворот вокруг оси v_1, затем поворот вокруг оси v_2. Эквивалентная ось вращения — $u_{1\to2} \approx (0.2847,\ 0.9552,\ 0.0813)^T$.

Результат показан на рисунке 7, координаты вершин приведены в таблице 5.

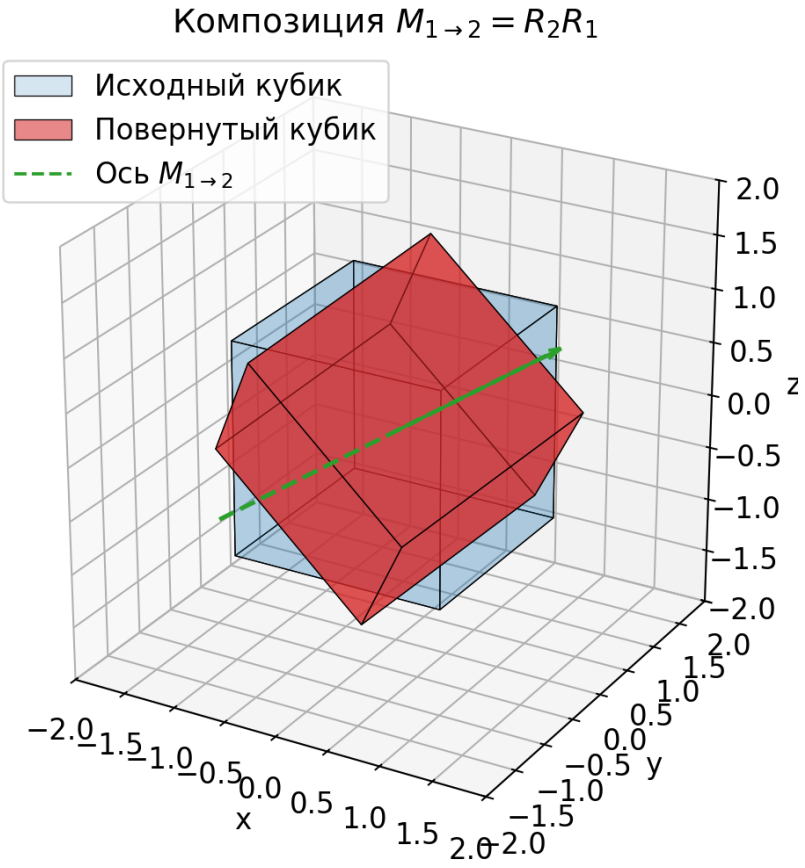


Рисунок 7 – Композиция поворотов: сначала R_1, затем R_2

Таблица 5 – Координаты вершин после применения $M_{1\to2} = R_2 R_1$

№ вершины	x	y	z
1	-0.674	-1.265	0.972
2	-1.653	-0.256	-0.451
3	-0.899	1.460	0.247
4	0.079	0.451	1.670
5	0.899	-1.460	-0.247
6	-0.079	-0.451	-1.670
7	0.674	1.265	-0.972
8	1.653	0.256	0.451

Композиция $M_{2\to1} = R_1 R_2$

Порядок преобразований: сначала поворот вокруг оси v_2, затем поворот вокруг оси v_1. Эквивалентная ось вращения — $u_{2\to1} \approx (0.6774,\ 0.5624,\ 0.4741)^T$.

Результат показан на рисунке 8, координаты вершин приведены в таблице 6.

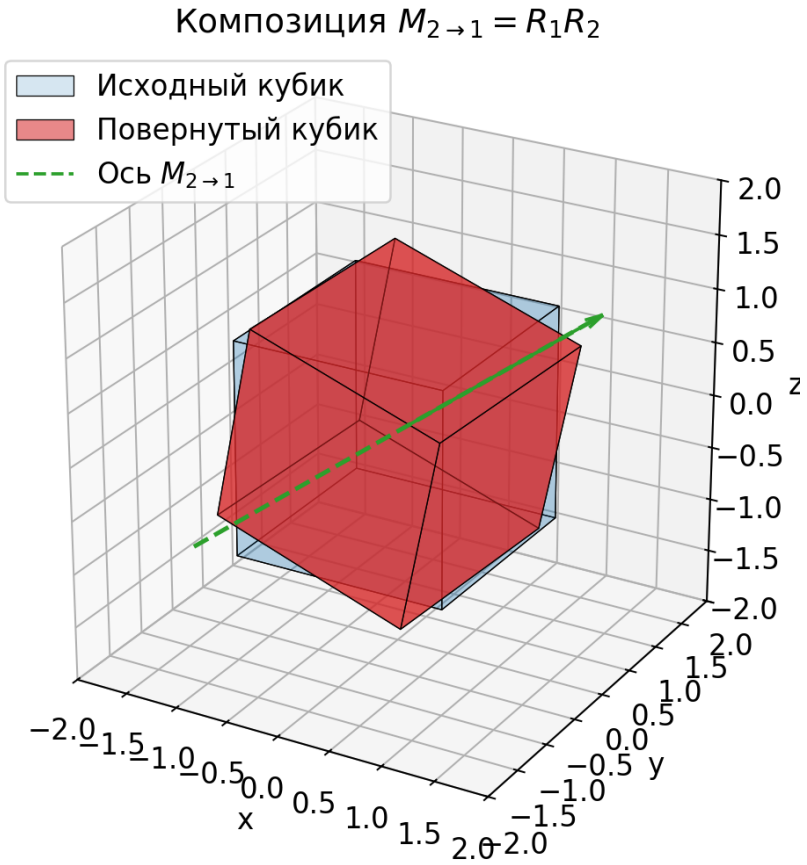


Рисунок 8 – Композиция поворотов: сначала R_2, затем R_1

Таблица 6 – Координаты вершин после применения $M_{2\to1} = R_1 R_2$

№ вершины	x	y	z
1	-1.330	-0.782	-0.787
2	-1.084	1.196	-0.628
3	-0.593	0.980	1.299
4	-0.839	-0.998	1.140
5	0.593	-0.980	-1.299
6	0.839	0.998	-1.140
7	1.330	0.782	0.787
8	1.084	-1.196	0.628

Анализ полученных результатов

Влияние оси вращения. Ось вращения определяет неподвижную прямую в пространстве: все точки, лежащие на ней, остаются на месте. Для оси v_1 это прямая

$$(0, 0, 0) + t(1, 1, 0),$$

а для оси v_2 — прямая

$$(0, 0, 0) + t(0, 1, 1).$$

Ось v_1 проходит через середины двух противоположных вертикальных ребер кубика, а ось v_2 — через середины двух противоположных ребер, параллельных оси x. Ни одна вершина кубика на этих осях не лежит, поэтому при выбранных углах вершины не остаются неподвижными.

Влияние угла поворота. Угол θ задает величину поворота вокруг выбранной оси:

- при $\theta = 0$ кубик остается неподвижным;
- при $\theta = \frac{\pi}{2}$ выполняется поворот на $90°$;
- при $\theta = \pi$ — поворот на $180°$;
- при $\theta = 2\pi$ кубик возвращается в исходное положение.

При выбранных углах $90°$ и $60°$ кубик не переходит в себя, а занимает новое положение. При этом преобразование является движением: расстояния между вершинами сохраняются. Например, расстояние между вершинами 1 и 2 после применения R_1 остается равным 2.

Влияние порядка преобразований. Матрицы $M_{1\to2} = R_2 R_1$ и $M_{2\to1} = R_1 R_2$ различны, поэтому итоговая ориентация кубика зависит от порядка поворотов. При этом

$$M_{2\to1} = R_1 M_{1\to2} R_1^{-1},$$

то есть матрицы подобны. Угол поворота определяется следом верхнего левого блока R размера 3×3:

$$\theta = \arccos\left(\frac{\operatorname{tr} R - 1}{2}\right).$$

Для обеих матриц $\operatorname{tr} R \approx -0.241025$. Поскольку следы подобных матриц совпадают, у обеих композиций одинаковый угол поворота $128.35°$, но эквивалентные оси различны:

$$u_{1\to 2} \approx \begin{bmatrix} 0.2847 \\ 0.9552 \\ 0.0813 \end{bmatrix}, \qquad u_{2\to 1} \approx \begin{bmatrix} 0.6774 \\ 0.5624 \\ 0.4741 \end{bmatrix}.$$

Таким образом, при одном и том же суммарном угле поворота кубик оказывается в разных положениях.

Проверка корректности матриц поворота. Для всех четырех матриц верхний блок 3×3 ортогонален:

$$R^T R = I,$$

а его определитель равен единице:

$$\det R = 1.$$

Это означает, что преобразования сохраняют длины, углы, объемы и ориентацию пространства. Кроме того, для осей выполняются равенства

$$R_1 \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \qquad R_2 \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

Здесь направления осей записаны в однородных координатах с нулевой четвертой компонентой. Следовательно, векторы v_1 и v_2 действительно являются собственными векторами соответствующих матриц с собственным числом 1, то есть задают оси вращения.

Восстановление оси вращения по матрице поворота

Для композиции поворотов эквивалентную ось можно восстановить непосредственно из матрицы. Если

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix},$$

то вектор оси вращения пропорционален вектору

$$u \sim \begin{bmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{bmatrix}.$$

После нормировки

$$u = \frac{u}{\|u\|}.$$

Именно так в программе получены оси $u_{1\to2}$ и $u_{2\to1}$. Формула применима при $\sin\theta \neq 0$; для углов 0 и π вектор оси находится через собственный вектор матрицы R с собственным числом 1.

Вывод по заданию 4

В задании построена матрица поворота вокруг произвольной оси, проходящей через центр кубика. Матричная экспонента $e^{J\theta}$ сведена к формуле Родригеса, а для осей x, y и z получены частные случаи R_x, R_y, R_z.
Для двух выбранных осей и углов построены матрицы R_1 и R_2, а также две их композиции — R_2R_1 и R_1R_2. Расчеты и результаты программы показывают, что матрицы композиций различны: при одинаковом угле $128.35°$ оси вращения не совпадают. Следовательно, порядок применения поворотов влияет на итоговую ориентацию кубика.
Корректность матриц подтверждена проверками: $R^TR = I$, $\det R = 1$, а векторы v_1 и v_2 остаются неподвижными. На всех графиках дополнительно показан вектор, коллинеарный оси вращения.

5 Задание 5. Вращение кубика вокруг вершины

Общая формула поворота вокруг оси, проходящей через точку

Пусть ось вращения проходит через точку

$$M = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix}$$

и имеет направление v, а угол поворота равен θ. Чтобы свести такое вращение к уже известному повороту вокруг начала координат, выполним три аффинных преобразования.

1. Перенесем объект так, чтобы точка M совпала с началом координат:

$$T(-M) = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

2. Повернем объект вокруг начала координат вокруг оси с направлением υ на угол θ:

$$R_\upsilon(\theta).$$

3. Вернем точку M на место обратным переносом:

$$T(M) = \begin{bmatrix} 1 & 0 & 0 & x_0 \\ 0 & 1 & 0 & y_0 \\ 0 & 0 & 1 & z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Итоговая матрица преобразования имеет вид

$$M_{\text{pivot}} = T(M)\, R_\upsilon(\theta)\, T(-M).$$

Обозначим через R верхний левый блок 3×3 матрицы $R_\upsilon(\theta)$. Тогда

$$M_{\text{pivot}} = \begin{bmatrix} R & M - RM \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix}.$$

Действительно,

$$T(M) \begin{bmatrix} R & 0 \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix} T(-M) = \begin{bmatrix} R & M - RM \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix}.$$

Вектор $M - RM$ — это сдвиг, который компенсирует поворот точки M вокруг начала координат. Поэтому

$$M_{\text{pivot}} \begin{bmatrix} M \\ 1 \end{bmatrix} = \begin{bmatrix} M \\ 1 \end{bmatrix},$$

то есть выбранная точка оси остается неподвижной. Матрица R — это матрица поворота вокруг начала координат, построенная в задании 4.

Выбранная матрица преобразования

Выберем вершину кубика

$$M = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

В однородных координатах:

$$M = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Ось вращения зададим направлением

$$v = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \qquad \theta = \frac{\pi}{2}, \qquad u = \frac{v}{\|v\|} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}.$$

Ось проходит через вершину M и описывается уравнением

$$L(t) = M + tu = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + t \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}.$$

Матрица поворота вокруг этой оси на угол $\theta = \frac{\pi}{2}$:

$$R_v\left(\frac{\pi}{2}\right) = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрицы переносов:

$$T(-M) = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad T(M) = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

После перемножения получаем

$$M_{\text{pivot}} = T(M)\, R_v\left(\frac{\pi}{2}\right)\, T(-M) = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\approx \begin{bmatrix} 0.500000 & 0.500000 & 0.707107 & -0.707107 \\ 0.500000 & 0.500000 & -0.707107 & 0.707107 \\ -0.707107 & 0.707107 & 0.000000 & 1.000000 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Проверим неподвижность выбранной вершины:

$$M_{\text{pivot}} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Следовательно, вершина $M = (1, 1, 1)$ остается на месте. На той же прямой лежит вершина $(-1, -1, 1)$, поэтому она также неподвижна.

Результаты выполнения программы

На рисунке 9 показаны исходный кубик (синий, полупрозрачный), кубик после применения M_{pivot} (красный) и ось вращения, проходящая через вершину $M = (1, 1, 1)$.

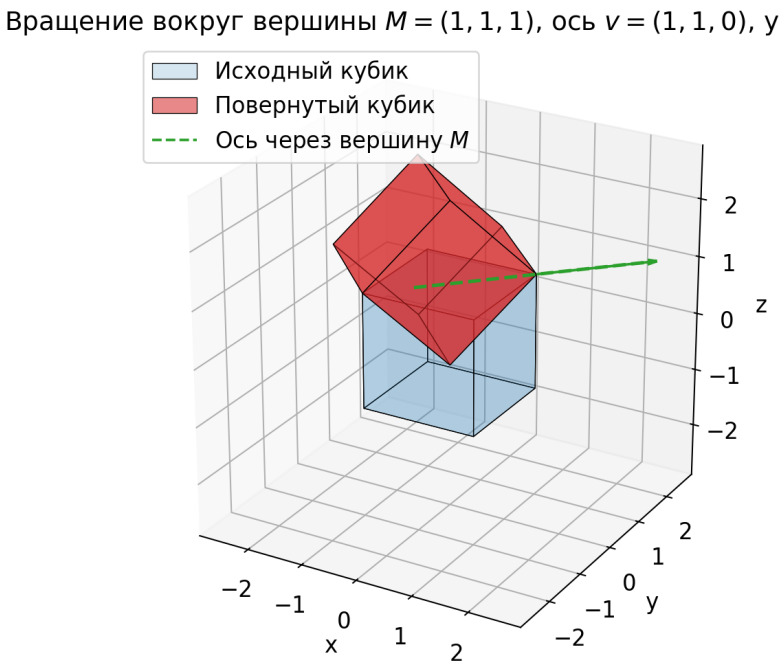


Рисунок 9 – Поворот вокруг оси через вершину $M = (1, 1, 1)$

Таблица 7 – Координаты вершин после применения M_{pivot}

№ вершины	x	y	z
1	-2.414	0.414	1.000
2	-1.414	1.414	-0.414
3	-0.414	2.414	1.000
4	-1.414	1.414	2.414
5	-1.000	-1.000	1.000
6	0.000	0.000	-0.414
7	1.000	1.000	1.000
8	0.000	0.000	2.414

Верхний блок матрицы ортогонален: $R^T R = I$, а его определитель равен единице: $\det R = 1$. Четвертая однородная координата всех вершин остается равной единице, поэтому преобразование является аффинным. Вершины 5 и 7 не меняют положения, поскольку лежат на оси вращения.

Вывод по заданию 5

Получена общая формула матрицы поворота вокруг оси с направлением v, проходящей через точку $M(x_0, y_0, z_0)$:

$$M_{\text{pivot}} = T(M)\, R_v(\theta)\, T(-M).$$

Для вершины $M = (1, 1, 1)$, оси $v = (1, 1, 0)$ и угла $\theta = \frac{\pi}{2}$ построена конкретная матрица M_{pivot}. Она переводит кубик в новое положение, оставляя выбранную вершину неподвижной. Результат подтвержден графиком и координатами вершин: преобразование является движением, а ось вращения проходит через вершину кубика.

Заключение по заданиям 1–5

Однородные координаты позволяют задавать масштабирование и перенос матрицами одного размера 4×4. Композиция выбранных масштабов восстанавливает размеры по x и z, сохраняя растяжение по y. Два переноса складываются, а порядок переноса и масштабирования влияет на положение образа: в ST масштабируется и смещение. Равенство определителей TS и ST показывает одинаковое изменение объёма, но не равенство преобразований. Повороты, вычисленные с помощью матричной экспоненты, сохраняют расстояния, углы и объём. Перестановка двух поворотов меняет эквивалентную ось, хотя угол в рассмотренном случае совпадает. Композиция $T(M)R_v(\theta)T(-M)$ позволяет вращать фигуру вокруг оси через заданную точку, сохраняя точки этой оси неподвижными.

Приложение А. Код блока А

```
"""ЛР3, блок А: задания 1–3, фигуры, масштабирование и перенос куба."""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Patch
from matplotlib.ticker import MaxNLocator
from mpl_toolkits.mplot3d.art3d import Line3DCollection, Poly3DCollection


def draw_shape(ax, vertices, faces, color, *, alpha=1.0):
    """Нарисовать грани фигуры с вершинами в столбцах матрицы 4×N."""
    # Из (x, y, z, w) получаем декартовы координаты (x/w, y/w, z/w).
    points = (vertices[:3, :] / vertices[3, :]).T
    if color == "none":
        # Контур рисуем линиями: пустые грани Poly3DCollection поддерживаются не везде.
        contours = [points[np.r_[face, face[0]]] for face in faces]
        ax.add_collection3d(Line3DCollection(
            contours, colors="0.45", linewidths=0.8, linestyles="--"
        ))
        return
    ax.add_collection3d(
        Poly3DCollection(
            points[faces], facecolors=color, edgecolors="k", linewidths=0.2,
            alpha=alpha,
        )
    )


vertices_cube = np.array([
    [-1, 1, 1, -1, -1, 1, 1, -1],
    [-1, -1, 1, 1, -1, -1, 1, 1],
    [-1, -1, -1, -1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 1, 1, 1],
])

faces_cube = np.array([
    [0, 1, 5, 4],
    [1, 2, 6, 5],
    [2, 3, 7, 6],
    [3, 0, 4, 7],
    [0, 1, 2, 3],
    [4, 5, 6, 7],
])

vertices_tetra = np.array([
    [-2,  4, 0,  1],  # x
    [-2, -4, 8,  1],  # y
    [ 0,  0, 0, 10],  # z
    [ 1,  1, 1,  1],  # w
])

faces_tetra = np.array([
    [0, 2, 1],  # основание
    [0, 1, 3],  # боковые грани
    [1, 2, 3],
    [2, 0, 3],
])

```

```
# Задание 2. Масштабы, выбранные Альбертом; четвёртая координата не меняется.
S1 = np.diag([2.0, 4.0, 10.0, 1.0])
S2 = np.diag([0.5, 1.0, 0.1, 1.0])
# Для векторов-столбцов сначала применяется правая матрица.
S21 = S2 @ S1

# Задание 3. Переносы, выбранные Альбертом, стоят в последнем столбце.
T1 = np.array([
    [1, 0, 0, 5],
    [0, 1, 0, 5],
    [0, 0, 1, 2],
    [0, 0, 0, 1],
], dtype=float)
T2 = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 3],
    [0, 0, 1, 10],
    [0, 0, 0, 1],
], dtype=float)
T21 = T2 @ T1
# Сравниваем первые выбранные матрицы переноса и масштаба.
TS = T1 @ S1
ST = S1 @ T1


def set_scene_axes(ax, *shapes):
    """Охватить все переданные фигуры, сохранив одинаковую единицу по осям."""
    points = np.hstack([vertices[:3, :] / vertices[3, :] for vertices in shapes])
    lower = points.min(axis=1)
    upper = points.max(axis=1)
    padding = 0.1 * (upper - lower)
    lower, upper = lower - padding, upper + padding
    ax.set_xlim(lower[0], upper[0])
    ax.set_ylim(lower[1], upper[1])
    ax.set_zlim(lower[2], upper[2])
    ax.set_box_aspect(upper - lower, zoom=0.85)
    ax.view_init(azim=-37.5, elev=30)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        axis.set_major_locator(MaxNLocator(nbins=4))


def plot_translation():
    """Переносы и их композиция: исходный куб показан контуром."""
    fig = plt.figure(figsize=(15, 5), layout="constrained")
    fig.suptitle("Задание 3. Перенос куба (исходный — контур)")
    cases = [
        ("T₁: (5; 5; 2)", T1),
        ("T₂: (0; 3; 10)", T2),
        ("T₂T₁: (5; 8; 12)", T21),
    ]
    images = [matrix @ vertices_cube for _, matrix in cases]
    for i, ((title, matrix), transformed) in enumerate(zip(cases, images), start=1):
        ax = fig.add_subplot(1, 3, i, projection="3d", proj_type="ortho")
        draw_shape(ax, vertices_cube, faces_cube, "none")
        draw_shape(ax, transformed, faces_cube, "blue")
        # Одинаковые пределы для трёх панелей делают сдвиги сравнимыми.
        set_scene_axes(ax, vertices_cube, *images)
        ax.set_title(title)
        print(f"\n{title}\n{matrix}")
        print("Центр куба:", transformed[:3, :].mean(axis=1))
```

```
    return fig


def plot_transform_order():
    """Сравнить перенос после масштаба и масштаб после переноса."""
    fig = plt.figure(figsize=(8, 7), layout="constrained")
    ax = fig.add_subplot(projection="3d", proj_type="ortho")
    vertices_ts = TS @ vertices_cube
    vertices_st = ST @ vertices_cube
    draw_shape(ax, vertices_ts, faces_cube, "tab:blue", alpha=0.55)
    draw_shape(ax, vertices_st, faces_cube, "tab:orange", alpha=0.55)
    set_scene_axes(ax, vertices_ts, vertices_st)
    ax.set_title("Задание 3. Порядок преобразований: T = T₁, S = S₁")
    ax.legend(handles=[
        Patch(facecolor="tab:blue", alpha=0.55,
              label="TS: масштаб → перенос; центр (5; 5; 2)"),
        Patch(facecolor="tab:orange", alpha=0.55,
              label="ST: перенос → масштаб; центр (10; 20; 20)"),
    ], loc="upper left", fontsize=9)
    for name, matrix, transformed in [("TS", TS, vertices_ts), ("ST", ST,
     ↪  vertices_st)]:
        print(f"\n{name}\n{matrix}")
        print("Центр куба:", transformed[:3, :].mean(axis=1))
        print("Размеры по x, y, z:", np.ptp(transformed[:3, :], axis=1))
    return fig


def plot_scaling():
    """Три результата: каждый отдельный масштаб и их последовательное применение."""
    fig = plt.figure(figsize=(15, 5), layout="constrained")
    fig.suptitle("Задание 2. Масштабирование куба")

    cases = [
        ("S₁: (2; 4; 10)", S1),
        ("S₂: (0,5; 1; 0,1)", S2),
        ("S₂S₁: (1; 4; 1)", S21),
    ]
    for i, (title, matrix) in enumerate(cases, start=1):
        transformed = matrix @ vertices_cube
        ax = fig.add_subplot(1, 3, i, projection="3d", proj_type="ortho")
        draw_shape(ax, transformed, faces_cube, "blue")

        # Свои пределы для каждого рисунка, но единичный масштаб по осям одинаков.
        # Куб центрирован в начале; все его вершины должны попасть в область.
        extent = np.max(np.abs(transformed[:3, :] / transformed[3, :]))
        limit = 1.15 * extent
        ax.set_xlim(-limit, limit)
        ax.set_ylim(-limit, limit)
        ax.set_zlim(-limit, limit)
        ticks = np.linspace(-extent, extent, 5)
        ax.set_xticks(ticks)
        ax.set_yticks(ticks)
        ax.set_zticks(ticks)
        ax.set_box_aspect([1, 1, 1], zoom=0.85)
        ax.view_init(azim=-37.5, elev=30)
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.set_zlabel("z")
        ax.set_title(title)

        print(f"\n{title}\n{matrix}")
        print("Размеры по x, y, z:", np.ptp(transformed[:3, :], axis=1))
    return fig
```

```
186
187
188 def main():
189     fig = plt.figure(figsize=(12, 6), layout="constrained")
190     ax = fig.add_subplot(1, 2, 1, projection="3d", proj_type="ortho")
191
192     draw_shape(ax, vertices_cube, faces_cube, "blue")
193
194     ax.set_box_aspect([1, 1, 1], zoom=0.85)
195     ax.set_xlim(-1, 1)
196     ax.set_ylim(-1, 1)
197     ax.set_zlim(-1, 1)
198     ax.view_init(azim=-37.5, elev=30)
199
200     ticks = np.linspace(-1, 1, 5)
201     ax.set_xticks(ticks)
202     ax.set_yticks(ticks)
203     ax.set_zticks(ticks)
204     ax.set_xlabel("x")
205     ax.set_ylabel("y")
206     ax.set_zlabel("z")
207     ax.set_title("Куб из методички")
208
209     ax_tetra = fig.add_subplot(1, 2, 2, projection="3d", proj_type="ortho")
210     draw_shape(ax_tetra, vertices_tetra, faces_tetra, "blue")
211
212     ax_tetra.set_xlim(-3, 5)
213     ax_tetra.set_ylim(-5, 9)
214     ax_tetra.set_zlim(-1, 11)
215     # Диапазоны осей 8, 14, 12: единичный масштаб одинаков по всем осям.
216     ax_tetra.set_box_aspect([8, 14, 12], zoom=0.85)
217     ax_tetra.view_init(azim=-37.5, elev=30)
218     ax_tetra.set_xlabel("x")
219     ax_tetra.set_ylabel("y")
220     ax_tetra.set_zlabel("z")
221     ax_tetra.set_title("Тетраэдр Альберта")
222
223     figures = {
224         "p01_figures": fig,
225         "p02_scaling": plot_scaling(),
226         "p03_translation": plot_translation(),
227         "p03_order": plot_transform_order(),
228     }
229     # Пути не зависят от рабочей папки: картинки черновика обновляются при запуске.
230     figures_dir = Path(__file__).resolve().parents[1] / "figures"
231     figures_dir.mkdir(parents=True, exist_ok=True)
232     for name, figure in figures.items():
233         path = figures_dir / f"{name}.png"
234         figure.savefig(path, dpi=200)
235         print(f"Сохранено: {path}")
236     plt.show()
237
238
239 if __name__ == "__main__":
240     main()
```

Листинг 1 – Отрисовка фигур, масштабирование и перенос куба

Приложение Б. Код заданий 4–5

```
"""ЛР3, задания 4-5. Вращение кубика вокруг оси и вокруг вершины.

Запуск: python block_45.py
Рисунки сохраняются в labs/lab03/figures/.
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
from scipy.linalg import expm

np.set_printoptions(precision=3, suppress=True)

FIGURES_DIR = Path(__file__).resolve().parent / "figures"

vertices_cube = np.array(
    [
        [-1, 1, 1, -1, -1, 1, 1, -1],
        [-1, -1, 1, 1, -1, -1, 1, 1],
        [-1, -1, -1, -1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
    ],
    dtype=float,
)

faces_cube = np.array(
    [
        [0, 1, 5, 4],
        [1, 2, 6, 5],
        [2, 3, 7, 6],
        [3, 0, 4, 7],
        [0, 1, 2, 3],
        [4, 5, 6, 7],
    ]
)


def normalize(v):
    """Возвращает единичный вектор, сонаправленный с v."""
    v = np.asarray(v, dtype=float)
    norm = np.linalg.norm(v)
    if np.isclose(norm, 0.0):
        raise ValueError("Нулевой вектор не может быть осью вращения.")
    return v / norm


def skew_matrix_4(u):
    """Кососимметрическая матрица 4x4 для нормированной оси u."""
    return np.array(
        [
            [0.0, -u[2], u[1], 0.0],
            [u[2], 0.0, -u[0], 0.0],
            [-u[1], u[0], 0.0, 0.0],
            [0.0, 0.0, 0.0, 0.0],
        ],
        dtype=float,
    )

```

```
61
62  def rotation_matrix(v, theta):
63      """Матрица поворота R_v(theta) = exp(J * theta)."""
64      u = normalize(v)
65      return expm(theta * skew_matrix_4(u))
66
67
68  def translation_matrix(t):
69      """Матрица переноса 4x4."""
70      T = np.eye(4)
71      T[:3, 3] = np.asarray(t, dtype=float)
72      return T
73
74
75  def rotation_about_point(v, theta, point):
76      """Вращение вокруг оси с направлением v, проходящей через точку point."""
77      p = np.asarray(point, dtype=float)
78      return translation_matrix(p) @ rotation_matrix(v, theta) @ translation_matrix(-p)
79
80
81  def rotation_axis_from_matrix(M):
82      """Восстанавливает единичную ось вращения по матрице поворота 4x4."""
83      R = M[:3, :3]
84      axis = np.array([R[2, 1] - R[1, 2], R[0, 2] - R[2, 0], R[1, 0] - R[0, 1]])
85      norm = np.linalg.norm(axis)
86
87      if norm < 1e-8:
88          eigvals, eigvecs = np.linalg.eig(R)
89          axis = np.real(eigvecs[:, np.argmin(np.abs(eigvals - 1.0))])
90          norm = np.linalg.norm(axis)
91
92      return axis / norm
93
94
95  def rotation_angle_from_matrix(M):
96      """Угол поворота по следу матрицы, в градусах."""
97      R = M[:3, :3]
98      cosine = (np.trace(R) - 1.0) / 2.0
99      return np.degrees(np.arccos(np.clip(cosine, -1.0, 1.0)))
100
101
102 def draw_shape(ax, vertices, faces, color, label, alpha=1.0, zorder=1):
103     """Рисует многогранник по однородным координатам вершин."""
104     points = (vertices[:3, :] / vertices[3, :]).T
105     collection = Poly3DCollection(
106         points[faces],
107         facecolors=color,
108         edgecolors="k",
109         linewidths=0.5,
110         alpha=alpha,
111         label=label,
112         zorder=zorder,
113     )
114     ax.add_collection3d(collection)
115
116
117 def draw_axis(
118     ax,
119     direction,
120     label,
121     color="tab:green",
122     length=2.0,
123     zorder=10,
```

```
    origin=(0.0, 0.0, 0.0),
):
    """Рисует ось вращения: пунктирную прямую и вектор из начала координат."""
    u = normalize(direction)
    o = np.asarray(origin, dtype=float)

    ax.plot([], [], [], color=color, linestyle="--", label=label)
    ax.plot(
        [o[0] - length * u[0], o[0] + length * u[0]],
        [o[1] - length * u[1], o[1] + length * u[1]],
        [o[2] - length * u[2], o[2] + length * u[2]],
        color=color,
        linestyle="--",
        linewidth=2.0,
        zorder=zorder,
    )
    ax.quiver(
        o[0],
        o[1],
        o[2],
        length * u[0],
        length * u[1],
        length * u[2],
        color=color,
        arrow_length_ratio=0.08,
        zorder=zorder,
    )


def plot_case(
    name,
    title,
    V_old,
    V_new,
    faces,
    axis_direction,
    axis_label,
    axis_origin=(0.0, 0.0, 0.0),
):
    """Строит и сохраняет график: исходный кубик, повернутый кубик и ось."""
    fig = plt.figure(figsize=(6.5, 6.5))
    ax = fig.add_subplot(projection="3d")
    ax.computed_zorder = False

    draw_shape(ax, V_old, faces, "tab:blue", "Исходный кубик", alpha=0.18, zorder=1)
    draw_shape(ax, V_new, faces, "tab:red", "Повернутый кубик", alpha=0.55, zorder=2)
    draw_axis(ax, axis_direction, axis_label, zorder=10, origin=axis_origin)

    points_old = (V_old[:3, :] / V_old[3, :]).T
    points_new = (V_new[:3, :] / V_new[3, :]).T
    extent = np.max(np.abs(np.vstack([points_old, points_new])))
    limit = 2.0 if extent <= 2.0 else extent + 0.5
    ax.set_xlim(-limit, limit)
    ax.set_ylim(-limit, limit)
    ax.set_zlim(-limit, limit)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")
    ax.set_title(title)
    ax.view_init(elev=25, azim=-60)
    ax.set_box_aspect([1, 1, 1])
    ax.legend(loc="upper left")

```

```
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    path = FIGURES_DIR / f"{name}.png"
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
    print(f"Сохранено: {path}")


def print_vertices(name, V):
    """Печатает декартовы координаты вершин после преобразования."""
    points = np.round((V[:3, :] / V[3, :]).T, 3)
    points[np.isclose(points, 0.0)] = 0.0
    print(f"Координаты после {name}:")
    print(points)
    print()


def check_rotation(name, M, axis):
    """Печатает проверки ортогональности, определителя и неподвижности оси."""
    R = M[:3, :3]
    p = np.append(normalize(axis), 1.0)
    print(f"{name}: det(R) = {np.linalg.det(R):.3f}")
    print(f"{name}: R^T R = I: {np.allclose(R.T @ R, np.eye(3))}")
    print(f"{name}: ось неподвижна: {np.allclose(M @ p, p)}")
    print()


def check_rotation_point(name, M, axis, point):
    """Проверки вращения вокруг оси, проходящей через заданную точку."""
    R = M[:3, :3]
    point = np.asarray(point, dtype=float)
    p = np.append(point, 1.0)
    d = np.append(normalize(axis), 0.0)
    expected_shift = point - R @ point
    print(f"{name}: det(R) = {np.linalg.det(R):.3f}")
    print(f"{name}: R^T R = I: {np.allclose(R.T @ R, np.eye(3))}")
    print(f"{name}: вершина {point} неподвижна: {np.allclose(M @ p, p)}")
    print(f"{name}: направление оси сохраняется: {np.allclose(M @ d, d)}")
    print(f"{name}: сдвиг M - R M: {np.allclose(M[:3, 3], expected_shift)}")
    print()


v1 = np.array([1.0, 1.0, 0.0])
theta1 = np.pi / 2.0

v2 = np.array([0.0, 1.0, 1.0])
theta2 = np.pi / 3.0

R1 = rotation_matrix(v1, theta1)
R2 = rotation_matrix(v2, theta2)

M_1_then_2 = R2 @ R1
M_2_then_1 = R1 @ R2

print("Матрица R1 = exp(J1 * theta1):")
print(R1)
print()
print("Матрица R2 = exp(J2 * theta2):")
print(R2)
print()

print("Матрица M_1_then_2 = R2 @ R1:")
print(M_1_then_2)
print()
```

```
print("Матрица M_2_then_1 = R1 @ R2:")
print(M_2_then_1)
print()

check_rotation("R1", R1, v1)
check_rotation("R2", R2, v2)

V1 = R1 @ vertices_cube
V2 = R2 @ vertices_cube
V_1_then_2 = M_1_then_2 @ vertices_cube
V_2_then_1 = M_2_then_1 @ vertices_cube

print_vertices("R1", V1)
print_vertices("R2", V2)
print_vertices("M_1_then_2 = R2 @ R1", V_1_then_2)
print_vertices("M_2_then_1 = R1 @ R2", V_2_then_1)

axis_1_then_2 = rotation_axis_from_matrix(M_1_then_2)
axis_2_then_1 = rotation_axis_from_matrix(M_2_then_1)

print("Ось вращения для M_1_then_2:", axis_1_then_2)
print("Ось вращения для M_2_then_1:", axis_2_then_1)
print("Угол M_1_then_2:", f"{rotation_angle_from_matrix(M_1_then_2):.2f}°")
print("Угол M_2_then_1:", f"{rotation_angle_from_matrix(M_2_then_1):.2f}°")
print()

check_rotation("M_1_then_2", M_1_then_2, axis_1_then_2)
check_rotation("M_2_then_1", M_2_then_1, axis_2_then_1)

plot_case(
    "p4A",
    r"Поворот вокруг оси $v_1 = (1, 1, 0)$, угол $90^\circ$",
    vertices_cube,
    V1,
    faces_cube,
    v1,
    "Ось $v_1$",
)
plot_case(
    "p4B",
    r"Поворот вокруг оси $v_2 = (0, 1, 1)$, угол $60^\circ$",
    vertices_cube,
    V2,
    faces_cube,
    v2,
    "Ось $v_2$",
)
plot_case(
    "p4AB",
    r"Композиция $M_{1 \to 2} = R_2 R_1$",
    vertices_cube,
    V_1_then_2,
    faces_cube,
    axis_1_then_2,
    r"Ось $M_{1 \to 2}$",
)
plot_case(
    "p4BA",
    r"Композиция $M_{2 \to 1} = R_1 R_2$",
    vertices_cube,
    V_2_then_1,
    faces_cube,
    axis_2_then_1,
```

```
    r"Ось $M_{2 \to 1}$",
)

vertex_index = 6
M_point = vertices_cube[:3, vertex_index]

v_pivot = np.array([1.0, 1.0, 0.0])
theta_pivot = np.pi / 2.0

M_pivot = rotation_about_point(v_pivot, theta_pivot, M_point)
V_pivot = M_pivot @ vertices_cube

print("Матрица M_pivot (вращение вокруг вершины):")
print(M_pivot)
print()

check_rotation_point("M_pivot", M_pivot, v_pivot, M_point)
print_vertices("M_pivot (вращение вокруг вершины)", V_pivot)
plot_case(
    "p5",
    r"Вращение вокруг вершины $M=(1,1,1)$, ось $v=(1,1,0)$, угол $90^\circ$",
    vertices_cube,
    V_pivot,
    faces_cube,
    v_pivot,
    r"Ось через вершину $M$",
    axis_origin=M_point,
)
```

Листинг 2 – Повороты вокруг оси через центр куба и через его вершину